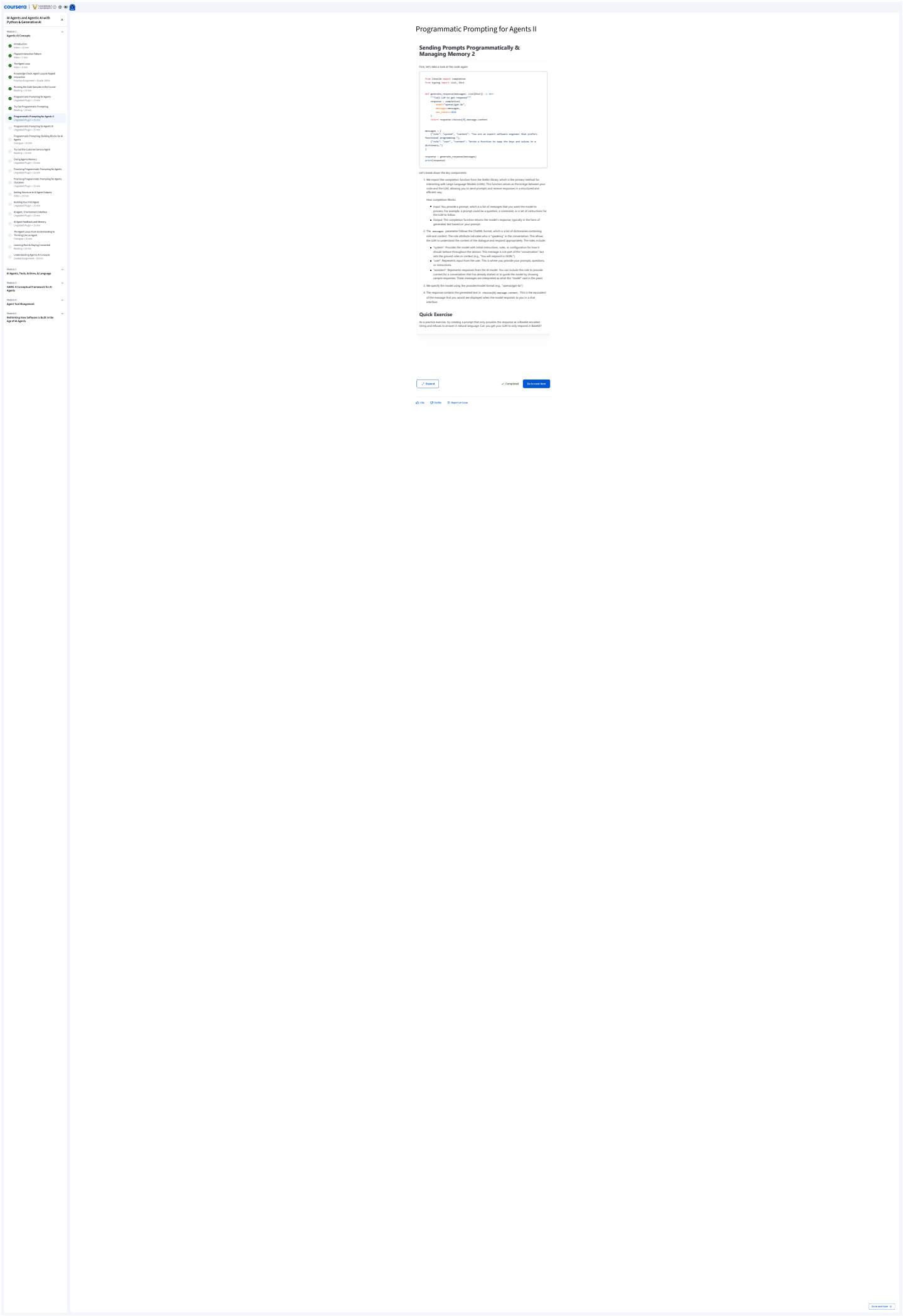

Programmatic Prompting for Agents II

Sending Prompts Programmatically & Managing Memory 2

First, let's take a look at the code again:

```
from litellm import completion
from typing import List, Dict


def generate_response(messages: List[Dict]) -> str:
    """Call LLM to get response"""
    response = completion(
        model="openai/gpt-4o",
        messages=messages,
        max_tokens=1024
    )
    return response.choices[0].message.content


messages = [
    {"role": "system", "content": "You are an expert software engineer that prefers
functional programming."},
    {"role": "user", "content": "Write a function to swap the keys and values in a
dictionary."}
]

response = generate_response(messages)
print(response)
```

Let's break down the key components:

1. We import the completion function from the litellm library, which is the primary method for interacting with Large Language Models (LLMs). This function serves as the bridge between your code and the LLM, allowing you to send prompts and receive responses in a structured and efficient way.

   How completion Works:

   - Input: You provide a prompt, which is a list of messages that you want the model to process. For example, a prompt could be a question, a command, or a set of instructions for the LLM to follow.
   - Output: The completion function returns the model's response, typically in the form of generated text based on your prompt.

2. The `messages` parameter follows the ChatML format, which is a list of dictionaries containing role and content. The role attribute indicates who is "speaking" in the conversation. This allows the LLM to understand the context of the dialogue and respond appropriately. The roles include:

   - "system": Provides the model with initial instructions, rules, or configuration for how it should behave throughout the session. This message is not part of the "conversation" but sets the ground rules or context (e.g., "You will respond in JSON.").
   - "user": Represents input from the user. This is where you provide your prompts, questions, or instructions.
   - "assistant": Represents responses from the AI model. You can include this role to provide context for a conversation that has already started or to guide the model by showing sample responses. These messages are interpreted as what the "model" said in the past.

3. We specify the model using the provider/model format (e.g., "openai/gpt-4o").

4. The response contains the generated text in `choices[0].message.content`. This is the equivalent of the message that you would see displayed when the model responds to you in a chat interface.

Quick Exercise

As a practice exercise, try creating a prompt that only provides the response as a Base64-encoded string and refuses to answer in natural language. Can you get your LLM to only respond in Base64?